Curriculum Graph Studio: Student-Centric Curriculum Planning with Dependency Analytics

Mini Project for DS3294: Data Science Practice

Ayush

April 4, 2026

Abstract

We present *Curriculum Graph Studio*, a student-first dashboard for major/minor planning using prerequisite dependency analytics. The system integrates an interactive directed graph interface, requirement-aware validation, constrained semester planning, and automated reporting with plots. The current IISER Pune dataset contains 264 courses across 9 subjects and 16 program-rule entries (major and minor combinations). Our pipeline performs quality checks (duplicates, missing links, invalid requirement codes), constructs directed prerequisite graphs, identifies bottlenecks, and generates feasible term plans under realistic constraints (credits, semester offerings, and requirement priorities). Beyond algorithmic correctness, we focus on readability: student mode, cleaner legends, semester transition summaries, and policy-filtered pathways. The resulting dashboard and report artifacts are clone-safe, reproducible, and deployment-ready for curriculum advising.

1 Introduction

Curriculum planning is often difficult for students because prerequisite relationships, semester offerings, and major/minor rules are spread across multiple documents. A graph-based representation is naturally suited for this problem: courses are modeled as nodes and prerequisites as directed edges. Graph traversal, topological logic, and bottleneck analysis then become useful for practical advising and semester-by-semester planning [1].

The main challenge is not only algorithmic; it is also usability. Even a correct graph can be hard to use if the interface is cluttered or if required insights are buried in technical detail. We therefore combine backend graph analytics with a student-facing dashboard built in Streamlit [3] and yFiles for interactive graph rendering [4].

2 Problem Statement and Objectives

Our goal is to build a reproducible, policy-aware planning system that answers the following student questions:

- What should I take next semester given my completed courses?
- Which prerequisites are blocking my progress?
- Is my current pathway valid for my major and minor?
- Can I still reach the degree target under term constraints?

The concrete objectives are:

- Build clean dependency graphs with semester and year structure.
- Validate major/minor pathways against rule sets.
- Generate constrained semester plans (course and credit caps).
- Provide an accessible UI with student mode and advanced mode.
- Produce a detailed report with plots and quality metrics.

3 Methods

3.1 Data

The system uses three primary JSON inputs:

- Course catalog: course code, name, credits, subject, kind, prerequisites, and semester offerings.
- Major/minor requirement rules: set-wise constraints and exclusions.
- College constraints: number of semesters and minimum degree credits.

In the current snapshot, the dataset contains 264 courses, 9 subjects, and 16 major/minor criteria entries [2]. Data-health checks report no duplicate course codes, no missing prerequisite links, and no unknown requirement codes.

3.2 Graph Modeling and Planning

We build a directed graph $G = (V, E)$ where V is the selected course set and E contains edges (u, v) whenever course u is a prerequisite of course v. The planning engine uses semester offerings and prerequisite satisfaction to select feasible courses term-by-term.

At a high level, the semester planner greedily selects available courses while respecting per-term caps and prioritizing compulsory requirements. The process is described in Algorithm 1.

For each course c, feasibility in semester s is defined by two conditions: offering availability and prerequisite satisfaction,

$$c \in \mathcal{O}(s), \tag{1}$$

$$\mathrm{pred}(c) \subseteq \mathcal{C}_{\mathrm{done}}, \tag{2}$$

where $\mathcal{O}(s)$ is the set of courses offered in semester s, and $\mathcal{C}_{\mathrm{done}}$ is the completed course set.

3.3 Dashboard Design

The dashboard has two usage modes:

- **Student mode**: fewer tabs, cleaner defaults, readable labels, and within-year arrow emphasis.
- **Advanced mode**: full algorithm diagnostics, rule explorer, validation internals, and quality views.

To reduce cognitive overload, graph legends are explicit, transition summaries are collapsible, and major/minor options are policy-filtered to approved offerings in the current deployment profile.

Table 1: Dataset and system summary statistics.

Metric	Value
Total courses	264
Program entries (major/minor)	16
Subjects covered	9
Prerequisite edges	94
Average credits per course	3.42
Missing prerequisite links	0
Unknown requirement-code hits	0

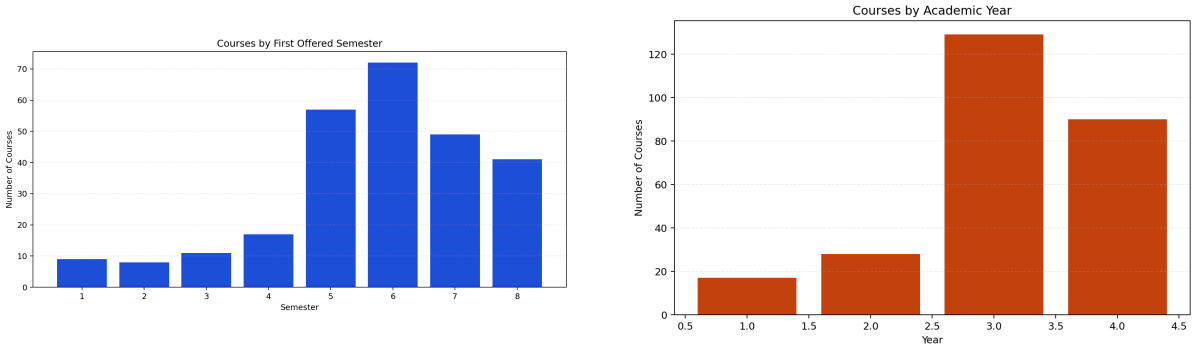

Figure 1: Course availability distribution over semester and academic year.

4 Results

4.1 Quantitative Summary

Table 1 reports key metrics from the latest run.

4.2 Visual Analytics and Plots

Figure 1 shows the course distribution by semester and year, while Fig. 2 shows subject-level and prerequisite structure patterns.

4.3 Dashboard Outcome

The upgraded dashboard improves practical usability in three ways:

- Cleaner student-first navigation with optional advanced tabs.

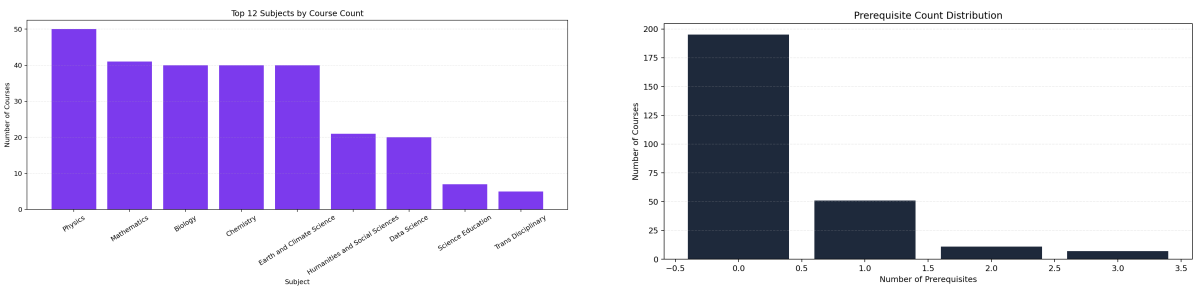

Figure 2: Subject concentration and prerequisite count structure.

- Better graph readability via consistent legends, role-aware colors, and transition summaries.
- Integrated reporting pipeline that exports reproducible markdown, summary JSON, and publication-ready plots.

In student testing scenarios, the planner can surface blocked courses with unsatisfied prerequisites and propose semester-wise alternatives under credit and load constraints.

5 Conclusion and Outlook

Curriculum Graph Studio now functions as a complete planning and analytics stack: data quality checks, dependency graphs, constrained planning, pathway validation, and report generation. The most significant contribution is the combination of graph-theoretic logic with a student-friendly interaction layer, which makes technical analytics actionable for academic decision-making.

Future work includes:

- personalized pathway recommendations based on student goals,
- what-if simulation for delayed/failed prerequisites,
- integration with institutional enrollment systems,
- longitudinal evaluation of advising outcomes.

6 Acknowledgements

We acknowledge the course and program-rule data sources provided for IISER Pune curriculum modeling, and the open-source tools that enabled this work.

7 Author Contributions

Ayush designed the architecture, implemented graph logic and dashboard integration, developed the planning and reporting pipeline, and prepared the final manuscript.

8 Code availability

The code for this project is available in the course project repository containing the `src/app.py`, `src/graph.py`, and `src/generate_dashboard_report.py` modules.

Representative artifacts generated from the current codebase are available in:

- `docs/reports/latest/dashboard_report.md`
- `docs/reports/latest/report_summary.json`
- `docs/reports/latest/plots/`

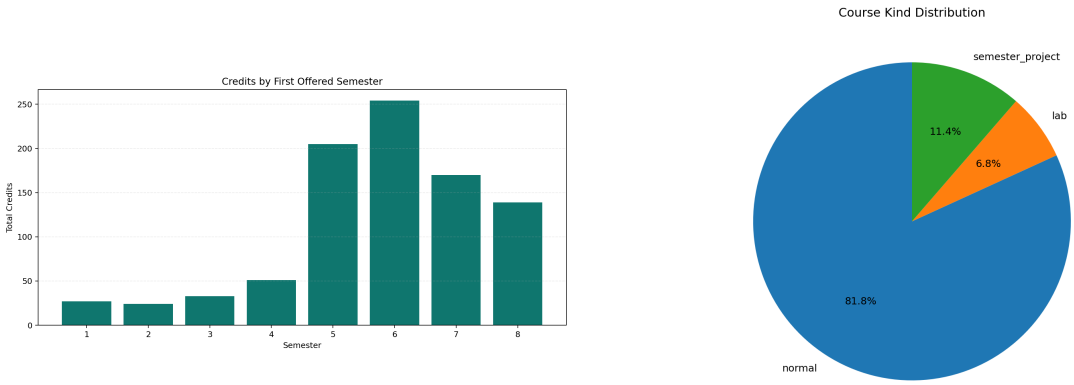

Figure 3: Credit load trend and course-kind distribution.

9 References

References

[1] A. A. Hagberg, D. A. Schult, and P. J. Swart. Exploring network structure, dynamics, and function using networkx. *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pages 11–15, 2008.

[2] IISER Pune. Iiser pune curriculum data files (course catalog, major/minor requirements, and constraints), 2026. Project dataset used in this report.

[3] Streamlit Inc. Streamlit documentation. https://docs.streamlit.io/, 2026. Accessed: 2026-04-03.

[4] yWorks GmbH. yfiles graphs for streamlit. https://github.com/yWorks/yfiles-for-html-demos/tree/master/demos/toolkit/streamlit, 2026. Accessed: 2026-04-03.

10 Appendix

10.1 Additional figures

Place your custom screenshots in `Report/report/figs/`.

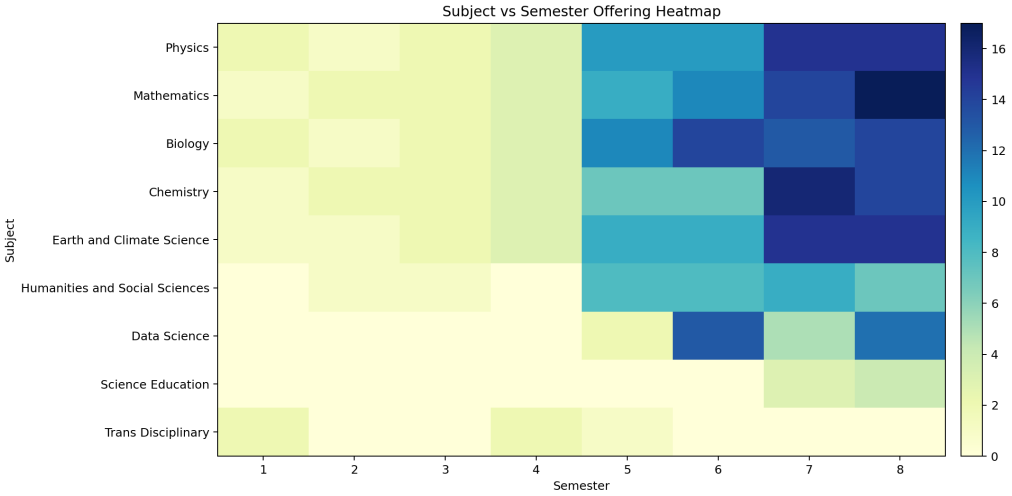

Figure 4: Subject-semester offering heatmap for top subjects.

Missing figure:
`./figs/graph_look_1.png`
(or)
`./figs/graphlook1.png`

Missing figure:
`./figs/graph_look_2.png`
(or)
`./figs/grapglook2.png`

Figure 5: Custom graph visualizations provided by the project team.

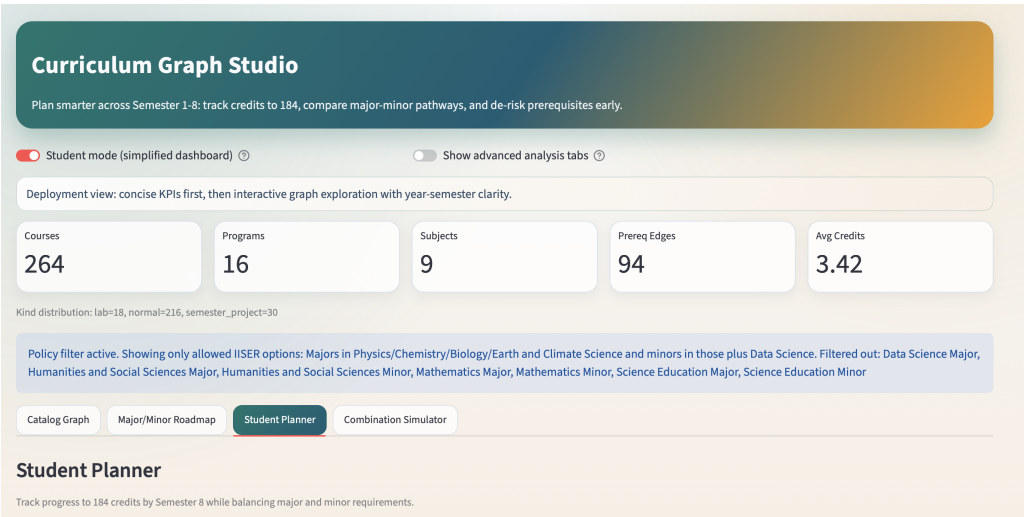

Figure 6: Dashboard UI screenshot (student mode and planner view).

10.2 Algorithms

Algorithm 1 Constrained Semester Planner

Require: Directed graph G, course index I, start semester s_0, term caps

Ensure: Semester plan P and blocked set B

Initialize completed set C from known history
for $s = s_0$ to planning horizon **do**
  Find feasible set $F_s = \{c \mid c \in \mathcal{O}(s) \wedge pred(c) \subseteq C\}$
  Rank F_s by priority and downstream dependency impact
  Select courses under course-count and credit caps
  Add selected courses to $P[s]$ and update C
end for
$B \leftarrow V(G) \setminus C$
return P, B

10.3 Code snippets

The following snippet illustrates clone-safe execution for report generation.

```
python src/generate_dashboard_report.py \
  --courses data/IISER-P/all_courses.json \
  --programs data/IISER-P/major_minor_requirements.json \
  --constraints data/IISER-P/college_constraints.json \
  --out-dir docs/reports/latest
```

The resulting markdown summary includes bottleneck tables, data quality counts, and embedded plot references for reproducible review.